

Role labels and call boundaries in code generation: a compression-free factorial study on 200 tasks

Daniil Privezentsev^{1*}

^{1*}National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

Role-based pipelines often change both instructions and model-call structure, obscuring the source of any efficiency gain. We cross neutral versus role-labelled stages with one versus three calls, holding stage operations and full supplied context fixed, and add direct solving. A reserved random sample of 200 Big-CodeBench tasks assigns 1,000 programs using GPT-5.4 mini with medium reasoning. Following a disclosed continuation restricted to untouched assignments, 996 programs are generated and checked with original native tests; 193 tasks pass reference controls. Quality ranges from 44.56% to 48.70%, avoiding the ceiling of the earlier study. The single-call role-minus-neutral quality difference is -1.05 percentage points (descriptive 95% interval [-4.71,+2.62]); the three-call difference is +1.04 [-3.11,+5.70]. None of four predeclared quality tests rejects after Holm adjustment. On complete resource pairs, single-call roles use 8.76% fewer tokens and have 18.45% lower API-equivalent valuation; three calls cost 2.09–2.50 times their single-call counterparts. Known primary usage is 8,585,185 tokens, valued at USD 14.20482 under published API rates; three interrupted calls lack usage. Separate development (400 programs) and an independent INoT adaptation (199 programs) are retained. Complete exposed traces and native reports support replay. The evidence identifies a protocol-specific resource difference, without establishing quality non-inferiority or a general benefit from internal agents.

Keywords: Code generation, role prompting, factorial design, token accounting, BigCodeBench, reproducibility

1 Introduction

Planning, implementation and review can be written as stages within one response or distributed over successive model calls. Calling those stages agents introduces a second change: role-specific wording. Comparisons that also summarize the input cannot determine which change explains a resource saving or a quality difference. In particular, a cheaper compressed response does not establish a benefit from internal role separation. This identification problem motivates a controlled test of labels and call boundaries using executable code tasks.

The intervention of interest is precise. Neutral stage numbers and the names planner, implementer and reviewer accompany identical operation sentences. Independently, those operations are requested in one fresh model turn or three fresh turns with the entire visible history forwarded. Every condition receives the same full task and supplied context. A direct one-call implementation instruction provides the simplest comparator. Thus the label contrasts isolate wording within a fixed topology, while the topology contrasts include intermediate information flow and repeated inference. Neither interpretation requires a claim that role names create independent latent agents.

The study separates a 40-task development stage from a 200-task reserved allocation, applies original executable BigCodeBench tests, and evaluates a separately frozen INoT PromptCode adaptation on the same reserved tasks. A one-issue SWE-bench demonstration supplies a limited repository-repair boundary. Quality has an opportunity to vary on these tasks, and every assigned cell remains visible even when generation or reference controls fail. Published GPT-5.4 mini tariffs permit an explicit resource valuation from actual subscription-channel counters, with a separate sensitivity to cache discounts.

The contributions are a compression-free 2×2 protocol with a direct comparator; a task-paired analysis with four predeclared quality contrasts and explicit missingness; and a complete exposed-response, program, native-report and token archive that can be audited without new model calls. The empirical claim is restricted to the frozen prompts, the requested model alias and the sampled tasks. The research question concerns whether role labels and call boundaries improve the measured quality–resource tradeoff under these conditions.

2 Related work

INoT describes compact PromptCode instructions for internal dialogue and reasoning [1]. This motivates evaluating a disclosed algorithm adaptation, while a separate minimal label intervention addresses whether named roles themselves change results. The two comparisons answer different questions: the INoT instruction changes the reasoning procedure, whereas the factorial label contrast preserves the operation sentences. The original Hybrid-INoT comparison additionally changed compression and reruns; its historical records are retained as background rather than pooled with the new experiment.

Single- versus multi-agent work under matched thinking-token budgets emphasizes that additional calls and additional computation are distinct interventions [2]. Here,

actual completion length is not held to a matched hard budget. Instead, token components and list-price valuations are reported as measured outcomes alongside quality. That design describes the implemented protocols’ tradeoffs and does not establish optimal allocation at a fixed inference budget.

Agentless uses a structured localization–repair–validation workflow [3]; SWE-agent couples the model to a repository interface [4]. Both introduce retrieval, tools and environment interaction beyond fixed-context function generation. Their published scores cannot be inserted into a paired matrix with different tasks and settings. Big-CodeBench [5] supplies diverse library-dependent tasks with executable tests, while SWE-bench [6, 7] measures repository changes against issue-specific tests. Native evaluation strengthens outcome validity, but passing a reference control does not settle every natural-language instruction/test disagreement.

Persona studies motivate a narrower claim than “roles improve reasoning.” Zheng et al. find that adding system personas does not reliably improve their factual-question evaluations [13]. Luz de Araujo et al. distinguish performance, robustness to irrelevant persona attributes, and fidelity to the assigned expertise; their results vary across models and tasks [14]. These are not tests of our coding prompts. They motivate changing labels while retaining the requested operations, and reporting the result for the exact labels used rather than for all possible personas.

Multi-agent evidence also depends on its comparator. Choi et al. attribute much of the gain in their NLP debate experiments to independent voting and show why communication must be separated from aggregation [15]. Conversely, Wunderlich et al. report configurations where debate or mixture-of-agents improve the compute–accuracy tradeoff relative to self-consistency on MMLU-Pro and BBH [16]. Together these findings support conditional evaluation, not a universal claim for or against multiple agents. Our neutral serial pipeline is not voting, a mixture of independent models, or a matched-budget ensemble. The contribution is a within-model factorial test on executable code with complete trace and cost accounting; general superiority over those alternatives remains outside its estimand.

3 Study design and estimands

3.1 A factorial intervention without compression

The main experiment crosses call topology (one or three fresh model turns) with stage labels (neutral or role-labelled). Table 1 defines the four factorial cells and the direct comparator. All structured conditions prescribe the same operations: analyze requirements and edge cases, implement a solution using the plan, then review and revise against the requirements. The label intervention replaces “stage 1”, “stage 2”, “stage 3” with “planner”, “implementer”, “reviewer”; the operation sentences remain unchanged. In a single turn, all three instructions precede the task. In three turns, each turn receives its stage instruction, the full task, the full supplied context, and every complete earlier exposed response. The final-output contract is identical: exactly one fenced Python program. The direct comparator receives only the implementation instruction and that contract.

Table 1 Implemented treatment matrix. The topology factor includes the exposure of intermediate responses across fresh calls.

Code	Calls	Stage labels	Operations
D	1	None	Direct implementation
SN	1	Neutral	Plan, implement, review
SR	1	Roles	Plan, implement, review
MN	3	Neutral	Plan, implement, review
MR	3	Roles	Plan, implement, review

Compression is absent in every cell. This replaces the earlier comparison of three uncompressed calls with one compressed call; it does not estimate the effect of the historical compressor. The isolated label contrast is a prompt intervention, not evidence of independent internal agents. The topology contrast includes intermediate information flow and repeated inference, rather than a purely mechanical call-count effect. The untested router and auxiliary checker are removed from the claimed method.

Let $Y_t(a)$ indicate that the single final program for task t in condition a passes the original benchmark tests in the validated environment. The four predeclared paired quality effects are

$$\Delta_{R,1} = \mathbb{E}_t[Y_t(\text{SR}) - Y_t(\text{SN})], \quad \Delta_{R,3} = \mathbb{E}_t[Y_t(\text{MR}) - Y_t(\text{MN})], \quad (1)$$

$$\Delta_{C,N} = \mathbb{E}_t[Y_t(\text{MN}) - Y_t(\text{SN})], \quad \Delta_{C,R} = \mathbb{E}_t[Y_t(\text{MR}) - Y_t(\text{SR})]. \quad (2)$$

Each null is a zero quality difference. The descriptive interaction is $\Delta_{R,3} - \Delta_{R,1}$. Expectations describe the sampled task population under the implemented protocol and its model sampling, not a provider-independent property. Available-case estimates require complete paired outcomes; with missingness they describe that observed subset unless stronger assumptions are made.

3.2 Task allocation and a separate development stage

The source is BigCodeBench v0.1.4 [5]. The main allocation contains 200 new task IDs from a previously frozen seeded random permutation of the reserved pool. The first 200 eligible IDs in that random order are taken after excluding eight dataset-card examples (/0–/7) incidentally exposed during license verification. This is neither a first-row convenience sample nor a difficulty-selected subset. Task IDs, exact model inputs and evaluator data are hashed before controls; no failed control causes replacement. The 40 development tasks are disjoint from these 200. Development uses two labelled repeats and 400 candidates; the main allocation uses one new repeat label and 1,000 candidates. The latter are 200 independent sampling units at most, not 1,000 independent tasks. Related benchmark tasks may further reduce effective independence. Public availability leaves training contamination unresolved.

All five conditions are assigned to every main task. Four disjoint index-modulo-four shards each contain 50 tasks. Within each shard, a frozen pseudorandom order determines task blocks and condition order; two workers process each shard, with at

most eight simultaneous experimental CLI processes. The complete allocation and source hashes are committed before dispatch. The model does not receive benchmark tests, reference solutions or test feedback during generation. Repeated labels identify fresh protocol observations; they are not provider sampling seeds.

3.3 Model execution and complete exposed traces

Generation uses the requested `gpt-5.4-mini` alias, medium reasoning, and official Codex CLI 0.153.4 through ChatGPT subscription authentication [10, 12]. The published model tariff makes resource valuation auditable; model choice is an economical available coding configuration, not a demonstrated global price minimum. The CLI is isolated from user configuration and repository instructions. Tools, browsing, delegation and test execution are disabled. Every call creates a fresh ephemeral thread in an empty working directory. The accepted event schema requires exactly one completed response and a valid usage record; unexpected tool, retry or compaction events invalidate the fixed text-only treatment.

Every submitted call retains its exact prompt, arguments, event stream, standard error and terminal status. Completed records also retain final text, token components and hashes of those original files. Multi-call prompts are reconstructed and byte-checked against the complete preceding exposed responses. No hidden provider reasoning is claimed to be available. The model alias does not fix immutable weights, and no verified temperature or model seed is exposed. A 180-second deadline applies to each turn. An input above 65,536 UTF-8 bytes is rejected rather than shortened. There is no matched hard output-token allowance; actual output length and resource consumption are outcomes of the treatment.

The original execution rule stops a shard after its first failed cell. Transport/deadline failures therefore leave unrelated assignments unsubmitted. A separately committed administrative amendment, recorded before held-out quality evaluation or inspection, permits only those never-submitted cells to continue. Any cell with even one started stage is permanently excluded from continuation. The exact pending list and terminal first-attempt archive are frozen before dispatch; prompts, settings, deadlines and original within-shard order are preserved. An explicit transport error or deadline ends a continuation cell without retry; quota, authentication or unclassified errors stop further dispatch. First attempts, continuations and missing outcomes remain distinguishable. Thus analysis follows predeclared contrasts under an amended execution protocol, not an unchanged preregistered run.

3.4 Native evaluation and eligibility controls

The unchanged BigCodeBench CLI at commit 09dd993f46c3 evaluates the original `instruct full` task tests. Each native run uses an offline, non-root, read-only container with a 3-GB memory cap, two CPUs and two evaluator workers. Prepared data, exact source tree, image identity, package versions, offline resources, commands and native reports are archived. Strict extraction selects the sole final Python block; an invalid final format yields an empty observed program and a format-failure flag, without alternative-answer selection or repair.

Before model generation, original gold programs and deliberately incorrect programs are tested for all 200 tasks. Dependency amendments are confined to this control phase, retain earlier failed attempts, and trigger fresh full-allocation controls. The final environment passes 193 gold controls and rejects all 200 incorrect controls. Seven tasks are consequently unavailable for quality measurement in every condition; their assignments and generation resources remain counted. The final image supplements the development environment with pinned dependencies and offline NLTK resources. Native gold diagnostics identify DNS failures (/101, /590), missing TensorFlow (/418), an unsupported encoder argument (/686), degenerate-moment assertions (/276), and archive/logging assertions (/14, /1028). A later read-only inventory confirms that the same image lacks the external commands used by the latter two tasks; attributing their assertions to those missing commands is a source-based diagnostic inference. No subsequent diagnosis changes the frozen eligibility gate. These checks establish evaluability and negative discrimination, not complete prompt–test semantic validity.

Only actual observed candidate programs are submitted for evaluation. Each native result is joined to the exact archived program, original task ID and verified environment; incomplete evaluator runs or inconsistent reports cannot supply quality observations. The raw native status is retained alongside a separate eligibility-gated analysis status. Missing generations are not fabricated native failures. The same eligibility gate applies to all conditions and the exploratory INoT replication.

3.5 Inference, missingness and resource valuation

For each of the four planned quality contrasts, the analysis reports complete-pair counts, both discordant counts, paired mean difference, exact two-sided McNemar binomial test and Holm-adjusted p across the four tests at familywise $\alpha = 0.05$. Percentile intervals use 10,000 task bootstrap resamples with analysis seed 20260908; these descriptive intervals are not simultaneous intervals. The interaction, resource contrasts and direct-solver comparisons remain descriptive. The interaction uses tasks complete across all four factorial conditions; it need not equal subtraction of two separately estimated label contrasts if their available-task subsets differ. Development repeats are averaged within task for its separate descriptive analysis and never pooled into the main tests.

Two hundred independent evaluable tasks would yield a worst-case single-proportion normal 95% half-width of approximately 6.9 percentage points. This is a precision rationale, not an 80%-power calculation for a paired effect; paired uncertainty depends on discordance. The study does not establish a two-point non-inferiority margin. Per-condition quality uses evaluable observations and reports all-assignment missingness ranges: unavailable outcomes are first all failures, then all successes. Those ranges are neither confidence intervals nor missing-at-random corrections.

Completed CLI turns expose input I , cached-input C and output O counters. Cached input is a subset of input; any separately exposed reasoning count is a subset of output. Neither is counted twice. Published standard API rates are USD 0.75, 0.075

and 4.50 per million tokens, respectively [11]. We report

$$V = \frac{0.75(I - C) + 0.075C + 4.50O}{10^6}, \quad V_0 = \frac{0.75I + 4.50O}{10^6} \quad \text{USD.} \quad (3)$$

V is a counterfactual API list-price valuation of measured subscription-channel usage, not a payment or invoice; V_0 is a no-cache sensitivity. Both exclude research-assistant work, software development, environment construction and evaluator compute. Candidate resource means sum their completed constituent turns. A separate inventory includes observed usage from interrupted candidates and explicitly counts turns without final usage; unknown usage is never zero-imputed. Resource ratios use the same complete paired task subset in numerator and denominator. Timing and cache state can influence valuation, especially across separately dispatched batches.

4 Exploratory INoT algorithm replication

The four constructed factorial cells do not stand in for a published method. We therefore add a separately frozen prompt-level replication of INoT [1], labelled INoT*. It uses the exact same 200 tasks, full supplied context, GPT-5.4 mini medium, final-output contract and native control gate. Each task receives one fresh call, with two workers and the same 180-second deadline. The independently worded PromptCode instruction describes two virtual debaters, initial answers, mutual arguments, critiques, rebuttals, updates and an agreement check, with a ten-round cap. This is conceptual guidance read by the model, not code executed by the host or a verified sequence of hidden agent interactions.

The stopping rule follows the source paper’s prose: agreement or the round cap. The corresponding source listing has an ambiguous loop condition and leaves the final answer unspecified if agreement is never reached. This replication explicitly returns the latest answer from virtual A in that case. Image augmentation is omitted for these text-only tasks. The exact independently worded instruction is archived. No verified executable author implementation was identified for this setup, and no author validation of the adaptation is claimed.

An independent administrative continuation follows the same never-submitted-only rule: the initial timeout is not rerun, and the remaining 137 untouched assignments are frozen and submitted once. All first-attempt and continuation traces remain separate. Comparisons of INoT* with D and SR are exploratory and outside the four primary tests. Task pairing controls membership, but the separate batch retains timing, cache-state and possible provider-drift differences. An observed result is evidence about this disclosed adaptation, not an exact reproduction of the original paper’s reported scores.

5 Primary results

5.1 Allocation, observed outcomes and missingness

All 1,000 assignments and 1,800 planned turns were submitted. The original attempt completed 457 of 460 submitted cells; the separately frozen continuation completed 539 of 540 untouched cells. Four cells remain incomplete: D on /1028 and SN on /388 after transport errors, SN on /249 after a deadline in the original attempt, and SN on /1028 after the continuation deadline. The final interrupted stream includes a completed response and usage, but its terminal status violates the 180-second acceptance criterion. It is therefore not used as an observed program. Event arrival times are not independently timestamped, and no timing sequence beyond the recorded terminal result is inferred.

The native evaluator checked all 996 observed programs. The common reference gate leaves 963 evaluable observations: 453 passes, 507 failures and three native timeouts. The other 33 observed programs belong to reference-ineligible tasks. Two final-format extraction failures, D /494 and MR /100, remain empty observed programs with their original responses retained. Generation failures, native timeouts, format errors and unavailable controls are distinct in the published records and in the complete task appendix.

Table 2 reports observed rates and extreme all-assignment ranges. The main five conditions span 44.56–48.70% test success, giving substantial room for both improvement and deterioration. The lower SN denominator follows its two missing generations on otherwise eligible tasks; both /1028 missing generations coincide with an unavailable reference control. The INoT* row belongs to a separate exploratory batch, discussed below.

Table 2 Main sample: 200 assignments per condition. Quality uses eligible observed programs; ranges retain all 200 assignments and are not confidence intervals. INoT* is a separate exploratory algorithm replication.

Arm	Generated	Passes	Rate (%)	Range (%)	Format errors
D	199	94/193	48.70	47.00–50.50	1
SN	197	93/191	48.69	46.50–51.00	0
SR	200	92/193	47.67	46.00–49.50	0
MN	200	86/193	44.56	43.00–46.50	0
MR	200	88/193	45.60	44.00–47.50	1
INoT*	199	96/192	50.00	48.00–52.00	0

5.2 Four predeclared quality contrasts

Table 3 and Fig. 1 show paired differences. Single-call roles produce six wins and eight losses against neutral stages on 191 eligible pairs. Three-call roles produce ten wins and eight losses on 193 pairs. Neither comparison provides evidence of improved

quality after correction. The two topology contrasts also fail to reject: the smallest adjusted p is 0.3134 for MN versus SN. The descriptive four-condition interaction is +2.09 percentage points, with interval $[-4.19, +8.38]$, on the common 191-task subset.

These findings do not establish equivalent quality. In particular, the interval for SR minus SN still includes a loss of 4.71 points. The primary data do not substantiate a claim of token savings with a guaranteed two-point quality tolerance. Failure to reject the other contrasts similarly leaves improvements and losses compatible with the observed uncertainty.

Table 3 Four predeclared quality comparisons under amended execution. Differences and descriptive 95% task-bootstrap intervals are percentage points. W/L are discordant pairs in favour of the first/second condition. Holm adjusts the exact two-sided McNemar tests across all four comparisons.

Contrast	Pairs	Difference [interval]	W/L	p	p_{Holm}
SR - SN	191	-1.05 $[-4.71, +2.62]$	6/8	0.7905	1.0000
MR - MN	193	+1.04 $[-3.11, +5.70]$	10/8	0.8145	1.0000
MN - SN	191	-4.71 $[-9.42, +0.00]$	6/15	0.0784	0.3134
MR - SR	193	-2.07 $[-6.22, +2.07]$	7/11	0.4807	1.0000

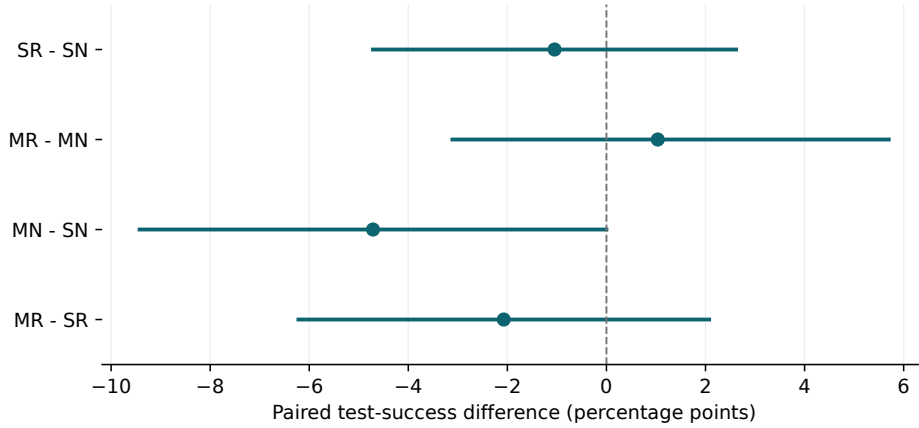


Figure 1 Paired quality differences in percentage points, with descriptive 95% task-bootstrap intervals. The reference line is zero; Holm-adjusted tests are reported separately in Table 3.

5.3 Measured resources and price sensitivity

Completed candidates use 8,569,456 tokens and have total valuation USD 14.1467676. The submitted-turn ledger additionally retains 15,729 known tokens from the interrupted continuation call. Across all submitted calls, known usage is therefore 8,585,185

tokens: 5,776,714 input, including 4,097,536 cached, and 2,808,471 output, including 2,076,742 reasoning. The 1,797 calls with counters are valued at USD 14.2048182, or USD 16.970655 without cache discounts. Three interrupted calls have unknown usage; the known total is not a complete account of consumption. None of these valuations is a subscription charge.

Table 4 reports per-condition means over completed candidates. The paired comparisons in Table 5 use a common subset for each resource contrast. SR versus SN uses 197 pairs: 8.76% fewer tokens and 18.45% lower valuation, with a mean USD difference of -0.0019761 [-0.0027617, -0.0012296]. The no-cache valuation remains 16.21% lower. Thus the observed label-associated resource difference does not disappear when the cache discount is removed, although its magnitude changes. These resource intervals are descriptive and outside the four quality tests.

Within three calls, the MR/MN valuation ratio is 0.985 and its mean-difference interval crosses zero. The single-call resource finding therefore does not generalize clearly across topologies. Conversely, MN/SN and MR/SR use 2.84 and 3.06 times as many tokens and cost 2.09 and 2.50 times as much; no-cache valuation ratios are 2.19 and 2.56. Full repeated inputs and intermediate responses form part of this measured expense. Figure 2 separates cached input, uncached input and output without double-counting reasoning.

Table 4 Resource means over completed candidates, including reference-ineligible tasks. Costs are API-equivalent USD; no-cache valuation removes the input cache discount. Interrupted-attempt usage is additionally reported in the submitted-turn ledger.

Arm	Candidates	Tokens	USD	No cache, USD
D	199	4 160	0.00655	0.00814
SN	197	5 122	0.01071	0.01230
SR	200	4 703	0.00888	0.01044
MN	200	14 578	0.02257	0.02714
MR	200	14 382	0.02222	0.02676
INoT*	199	3 858	0.00532	0.00670

Table 5 Descriptive paired resource contrasts. Ratios divide paired means; uncertainty is a 95% task-bootstrap interval for the paired mean difference in API-equivalent USD, not for the ratio.

Contrast	Pairs	Token ratio	USD ratio	USD difference [interval]
SR - SN	197	0.912	0.816	-0.0020 [-0.0028, -0.0012]
MR - MN	200	0.987	0.985	-0.0003 [-0.0016, +0.0008]
MN - SN	197	2.839	2.088	+0.0117 [+0.0104, +0.0130]
MR - SR	200	3.058	2.504	+0.0133 [+0.0121, +0.0147]

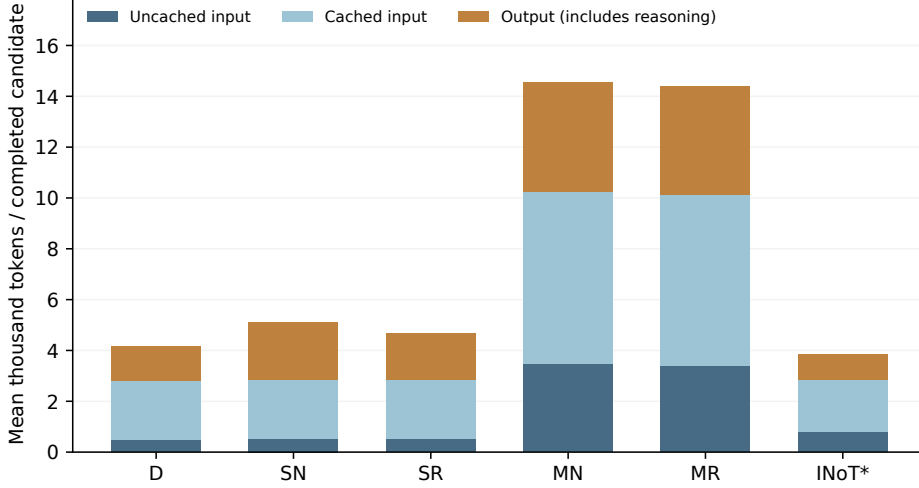


Figure 2 Mean token components per completed candidate. Cached tokens are separated from the remaining input; output includes reasoning. Denominators include reference-ineligible tasks. INoT* is a separately dispatched adaptation.

6 Exploratory comparators and development replication

Direct solving has the lowest observed token and valuation means among the five main conditions. SN minus D has a paired quality difference of zero $[-4.19, +4.19]$ on 191 tasks, but a valuation ratio of 1.632 on 197 resource pairs. MR minus D has a quality difference of -3.11 $[-7.77, +1.04]$ and costs 3.359 times as much on 199 resource pairs. These descriptive comparisons support keeping a direct baseline; they do not establish statistical dominance.

The independent INoT* adaptation completes 199 of 200 assigned programs, with one original deadline and no final-format extraction error. On 192 eligible outcomes, 96 pass (50%); all-assignment bounds are 48–52%. Known usage is 767,679 tokens and USD 1.05886035 valuation (USD 1.33326675 without cache discounts), with one call lacking usage. Relative to D, its paired quality difference is +1.04 $[-3.65, +5.73]$ and its valuation ratio is 0.805; relative to SR the corresponding values are +2.08 $[-2.08, +6.25]$ and 0.600. Table 6 retains different quality/resource pair counts. Because this algorithm adaptation ran in a separate batch, timing, cache and provider changes remain potential confounders. It is not an author-code replication or part of the four-test family. Figure 3 places the observed means on a common scale without declaring a statistically established frontier.

7 Separate development calibration

The development stage precedes the main sample and uses 40 disjoint randomly selected tasks, the same five conditions and two fresh labelled repeats (2 and 3). All 400

Table 6 Exploratory comparisons outside the four-test family. Intervals are descriptive 95% task bootstraps. Quality/resource columns give their respective complete-pair counts. INoT* was dispatched in a separate batch.

Contrast	Pairs Q/R	Quality difference [interval]	Tokens, ratio	USD, ratio
SN - D	191/197	+0.00 [−4.19, +4.19]	1.230	1.632
MR - D	193/199	−3.11 [−7.77, +1.04]	3.444	3.359
INoT* - D	192/198	+1.04 [−3.65, +5.73]	0.925	0.805
INoT* - SR	192/199	+2.08 [−2.08, +6.25]	0.821	0.600

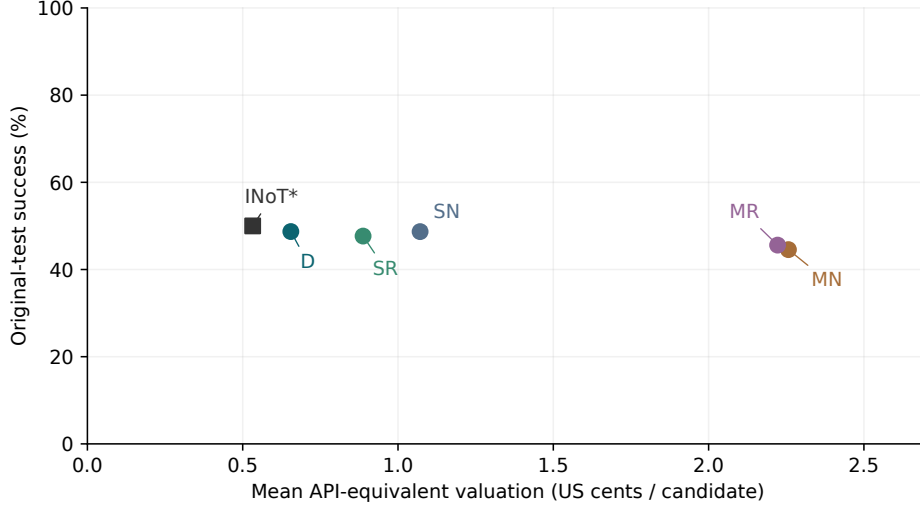


Figure 3 Observed test-success rates and mean API-equivalent valuation. The full 0–100% quality axis avoids exaggerating small rate differences. These points have different eligible denominators; paired uncertainty is provided in Tables 3 and 6.

assigned programs and 720 CLI turns complete and pass the trace audit. The full 400 native records reconcile to the exact exported programs. Reference task /1005 fails in the development environment, leaving 390 evaluable attempts over 39 tasks; its ten generation costs remain included. No final-format extraction fails. Each attempt is scored separately, with no best-of-two selection. For paired development contrasts, the two repeats are averaged within task before resampling.

The observed rates range from 43.59% to 53.85%. D passes 42/78 attempts, SR and MN each 39/78, MR 36/78, and SN 34/78. D also has the smallest resource mean. Complete generation uses 3,703,821 tokens, valued at USD 6.6113451, or USD 7.7687595 without cache discounts. The denominator remains 40 task clusters for resources and 39 for quality contrasts.

The role-minus-neutral development differences are +6.41 percentage points in one call and −3.85 in three calls. The difference of differences is −10.26 points, with a descriptive task-bootstrap interval of [−19.23, −1.28]. Three-call token means are

Table 7 Development results: 80 assigned candidates per condition, two repeats on 40 tasks. Quality uses 78 evaluable attempts; resource means use all 80. Missingness ranges use the full assignment denominator and are not confidence intervals.

Arm	Passes	Rate (%)	Range (%)	Tokens	USD	No cache
D	42/78	53.85	52.50–55.00	4 500	0.00776	0.00939
SN	34/78	43.59	42.50–45.00	5 598	0.01251	0.01417
SR	39/78	50.00	48.75–51.25	5 189	0.01070	0.01234
MN	39/78	50.00	48.75–51.25	15 636	0.02624	0.03090
MR	36/78	46.15	45.00–47.50	15 375	0.02543	0.03030

Table 8 Paired development contrasts. Quality differences and descriptive 95% task-bootstrap intervals are percentage points over 39 complete task clusters; resource ratios use 40. The last two comparisons are exploratory. No confirmatory p-value or non-inferiority test is assigned.

Contrast	Quality difference [interval]	Token ratio	USD ratio
SR - SN	+6.41 [+1.28, +11.54]	0.927	0.855
MR - MN	−3.85 [−11.54, +2.56]	0.983	0.969
MN - SN	+6.41 [−2.56, +15.38]	2.793	2.097
MR - SR	−3.85 [−8.97, +1.28]	2.963	2.377
SN - D	−10.26 [−19.23, −2.56]	1.244	1.613
MR - D	−7.69 [−17.95, +1.28]	3.417	3.277

2.79 and 2.96 times the corresponding single-call means; valuation ratios are 2.10 and 2.38. These development observations motivated the separate reserved-task allocation. They are not additional observations in its four-test family and cannot replace a main-sample result. Complete two-repeat outcomes are retained in the appendix and public archive.

8 Illustrative discordant cases

The following four examples were selected after generation by the frozen deterministic rule: for each label contrast, choose the lowest numeric task ID in each direction in which both programs were observed and eligibility-gated, retaining one role-labelled–neutral discordance in each direction. They are descriptive examples outside the four primary tests. They do not estimate discordance frequencies or identify a hidden reasoning mechanism.

For **BigCodeBench/428**, the role-labelled single-call program passed while the neutral program failed. Both programs performed the requested outer merge and scaling of numeric features from `df1`. The role-labelled program scaled a copy of `df1` before one merge and preserved the left-dataframe columns first. The neutral program merged first, then dropped and reinserted the scaled columns in a second merge, producing

the order `id`, `feature4`, `feature5`, `feature1`, `feature2`, `feature3`. The evaluator’s `test_case_1` required `id`, `feature1`, `feature2`, `feature3`, `feature4`, `feature5` and failed with that list mismatch. The task prose specifies the operations but does not explicitly specify column order; the test supplies a stricter output contract.

For `BigCodeBench/71`, the neutral single-call program passed while the role-labelled program failed. The role-labelled implementation used sample standard deviation, `Series.std(ddof=1)`, while the neutral implementation and canonical solution used population standard deviation, `np.std` with `ddof=0`. The role-labelled program consequently failed `test_case_2`, `test_case_3`, `test_case_4`, and `test_case_5`; the recorded discrepancies were respectively 11.900380145988935 versus 11.017611134090766, 8.568005268772557 versus 8.01463505095522, NaN versus 0.0, and 47.95684491664328 versus 46.07544623291379. “Standard deviation” is underspecified in the prose, while the executable contract fixes the population convention.

For `BigCodeBench/167`, the role-labelled three-call program passed while the neutral program failed. The neutral implementation selected `max(2,min(5,num_types))` stacked segments. It therefore produced 50 patches for `num_types=10` and 2 for `num_types=1`, whereas the role-labelled implementation used exactly `num_types` segments and produced 100 and 1. The native failures were `test_case_3`, `50  $\neq$  100`, and `test_case_4`, `2  $\neq$  1`. The prose describes categories and segmented values but leaves the segment count open; the tests require it to equal `num_types`.

For `BigCodeBench/31`, the neutral three-call program passed while the role-labelled program failed. The neutral implementation preserved the first-occurrence order of the `Counter`; the role-labelled implementation sorted by descending frequency and then lexicographically. On the test text containing `$is` three times, `$second` once, and `$sentence` twice, the evaluator expected values 3, 1, 2 in first-occurrence category order. Sorting moved `$sentence` ahead of `$second`, and `test_case_2` reported “Expected value ‘1.0’, but got ‘2’.” The prose requires a frequency plot without specifying category order, so this is an executable-order mismatch rather than evidence of a causal role effect.

All descriptions use the archived prompts, programs, and native assertions. They describe observed code behavior only; they do not infer private reasoning, latent agents, or representative population frequencies.

9 Repository-patch demonstration and an explicit incomplete run

A separate one-task SWE-bench Lite development demonstration uses `pvlb__pvlb-python-1606`. Deterministic BM25 retrieval from the exact base commit selects whole eligible Python files within 24,000 source bytes; nine files totaling 23,863 bytes enter the identical packet for all arms. Test files, hints and reference patches are excluded from retrieval. Files too large to fit are listed, not truncated. This is an explicitly selected packet, not the full repository, and omitted relevant files remain a retrieval limitation.

The unchanged pinned SWE-bench harness initially fails even with the gold patch because its official task image contains an incompatible NumPy version. A separate image changing NumPy to 1.26.4 passes all 11 gold tests; an applicable comment-only negative passes ten tests and fails the regression test. All initial failures and both image identities are retained. These controls validate the adjusted environment, not the unchanged release image. The controller runs offline, while the unchanged upstream child task container retains its default Docker network; no credentials or personal directories are mounted.

The fixed ten-candidate generation stops after one CLI call exceeds the 180-second deadline. Two complete patches remain: D applies but fails the target regression test (ten other tests pass); SN is rejected by native patch application. Eight assigned candidates remain missing. Both observed evaluations pass the input/patch/report audit and have no identified infrastructure failure. Four CLI turns were submitted; three expose complete usage totaling 56,796 tokens and USD 0.1379895 in API-equivalent valuation, while one has unavailable usage. This is a partial development demonstration with no resolved instance, not a Lite benchmark score. It replaces the old surrogate `check()` route with real patch application and tests while retaining the failure to complete the planned matrix.

10 Discussion

The controlled label contrast yields a narrower finding than the original package-level efficiency claim. Within one response, role names accompany lower measured resource use while the quality interval remains compatible with a meaningful loss. Within three calls, neither a clear resource reduction nor improved quality is established. The result is consistent with prompt wording changing generated length; the archive does not identify a latent cognitive mechanism or establish why that change occurs.

The primary quality pattern also differs from development: the development single-call role difference was positive and its three-call counterpart negative, whereas the main point estimates reverse those directions. The main interaction interval includes zero. Retaining the separate reserved analysis prevents a favorable development pattern from becoming the final evidence by selection. Meanwhile, the increased resource requirement of three calls appears in both stages, strengthening the descriptive observation for this implementation without proving that multi-call systems cannot help other tasks.

An algorithmic instruction such as INoT* and a minimal role-label substitution are different interventions. The exploratory adaptation has a favorable observed resource mean, but its quality interval and separate execution batch prevent a superiority claim. The four deterministic cases further show that small implementation conventions can determine benchmark success when natural-language prose is underspecified. Their original outcomes remain in the analysis. Accordingly, the endpoint is success against the retained executable contract, not unrestricted correctness or evidence of independent internal agents.

11 Validity and limits of generalization

The study tests one requested model alias, one reasoning setting and one CLI version on a sampled function-generation benchmark. The repeated development stage does not supply model-level replication of the main sample. Public task availability leaves training contamination unresolved; related tasks can also violate an independent-task approximation. A different model, language, task distribution or service revision could reverse an observed contrast. Recorded dates and runtime hashes identify the executed client, not immutable provider weights.

The deadline and administrative continuation create a second boundary. Failed generations remain missing, and the amendment is disclosed. Available-pair inference is about the observed eligible subset. Assignment-denominator ranges show extreme missing-outcome possibilities without correcting selection bias. A timeout could reflect computational difficulty rather than random transport noise, so failure to reject a quality difference does not prove equivalent quality. Neither the sample size nor the analysis establishes a two-percentage-point non-inferiority margin.

The interventions concern explicit instructions. Role labels do not certify independent agents, and call boundaries also expose intermediate responses. Although task information and stage operations are controlled, there is no matched hard output-token allowance. Resource comparisons include induced reasoning length, repeated context transmission and client overhead. List-price valuation depends on cache state; the no-cache sensitivity removes the discount but does not make subscription execution identical to an API deployment. Research and evaluator compute are excluded and no total cost of ownership is estimated.

Native tests improve auditability while retaining benchmark limitations. Gold and incorrect controls detect an unusable environment or an insensitive negative check, but passing those controls does not prove that every original assertion implements the natural-language task. A separate development audit records instruction/test disagreements as annotations, with original prompts, tests and scores unchanged. Those annotations are not post-hoc exclusions or a replacement gold standard. The seven main reference-ineligible tasks likewise remain in the assignment and resource accounting.

The INoT comparison is a disclosed prompt-level adaptation with an explicit resolution of source ambiguity. Its separate batch retains timing and cache differences. Native author INoT, Agentless and SWE-agent implementations are not evaluated. The latter systems introduce retrieval, tools and repository interaction outside this fixed-context factorial question. The one-issue SWE-bench demonstration provides genuine patch and test evidence, but its incomplete matrix cannot support a repository-benchmark score or comparative agent ranking. A native multi-repository comparison and a matched-budget, multi-model replication remain necessary for those broader claims.

12 Conclusion

An uncompressed factorial experiment on 200 reserved tasks replaces the earlier confounded comparison with 996 native-checked programs. Single-call role labels reduce

measured resources on complete pairs; quality non-inferiority remains unproven. Three calls require substantially more resources without a detected quality benefit in the four planned tests. A separate 400-program development stage, 199-program INoT adaptation, full task outcomes and exact exposed traces make both the findings and their boundaries auditable. The supported contribution is a controlled, reproducible measurement of these implemented protocols, not a general claim that internal roles improve reasoning.

Data and code availability

The repository [9] retains all prompts, exposed answers, programs, native reports, assignments, source/environment identities and per-call counters. The primary evidence snapshot is commit 96be37745b7f, directory 20260908_codex_mini_heldout200. A SHA-256 inventory covers 12,082 files; Git blob checks preserve exact original bytes. Public-archive replay reconstructs all 996 primary records and their summary byte for byte in an isolated analysis environment; automated Linux checks also cover the separate development and INoT archives. Replay validates retained observations and computations without new model calls; it does not claim deterministic provider re-execution. The exact prompt appendix and decision timeline expose the treatment and administrative amendment. Third-party materials retain their source licenses.

Declarations

Funding and competing interests: no external funding or competing interest was reported in the preceding manuscript. **Research scope:** public software benchmarks; no human-participant study. **Responsibility:** AI assisted software, audits and revision; the author remains responsible for scientific claims. Software tests are not experimental model observations.

A Exact executed prompts

The following English strings are used in both language editions because they are the executed experimental instructions. Typographic wrapping is for the page only; exact UTF-8 prompts and forwarded responses are retained in the archive.

Common instruction.

Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available.

Factorial operations.

In order, the three exact operation sentences are:

Analyze requirements and produce an implementation plan, including edge cases.

Implement the requested program or repository change using the plan.

Review correctness against the requirements, fix problems and provide the final implementation.

Each sentence is prefixed by its neutral label (stage 1/2/3) or role label (planner/implementer/reviewer), followed by a colon and a space. Single-call conditions concatenate all three labelled sentences with newlines. Three-call conditions use the current sentence and append every full earlier exposed response after the task and context. The direct condition uses the following sentence alone before the common final contract:

Implement the requested program.

Final-output contract.

Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available.

The contract appears in each single-call condition and in stage 3 of each three-call condition. Task text is introduced by TASK; supplied context by FULL SUPPLIED CONTEXT. Each earlier response is introduced by PREVIOUS STAGE i (COMPLETE OUTPUT), with one-based i . The runner enforces the byte guard before dispatch.

A.1 Independent INoT instruction

```
<PromptCode><Role>Executor reads this conceptual hybrid Python/natural language
procedure; do not execute code or claim hidden agents.</Role> <Reasoning-
Logic>Initialize virtual A and B for the task; obtain result_A, thought_A and result_B,
thought_B. Set agreement=False. for round in 1..10: argument_A/B; critique_A
critiques B and critique_B critiques A; rebuttal_A/B answer the opposite critique;
result_A,thought_A=update(rebuttal_B); result_B,thought_B=update(rebuttal_A);
agreement=(result_A==result_B); if agreement: break. final_result=result_A if agree-
ment else result_A (latest A fallback). </ReasoningLogic></PromptCode> Return
final_result in the requested format, with no tools, tests, unavailable evidence, or
hidden-agent claims. Omit image augmentation for this text task.
```

For INoT*, this instruction precedes the full task and context; the same final-output contract follows them. The common execution instruction is also retained. This is model-read conceptual guidance, not executable host code.

B Complete primary task outcomes

Table 9 All main task outcomes, part 1/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

ID	D	SN	SR	MN	MR	INoT*
14	U	U	U	U	U	U
16	P	P	P	P	P	P
24	P	P	P	P	P	P
31	F	F	F	P	F	P
33	P	P	P	P	P	P
35	F	F	F	F	F	F
37	P	P	P	P	P	P
38	P	P	P	P	P	P
48	P	P	P	P	P	P
49	F	F	F	F	F	F
71	F	P	F	F	F	P
75	F	F	F	F	F	F
97	P	P	P	P	P	P
100	F	F	F	F	F	F
101	U	U	U	U	U	U
105	P	P	P	P	P	P
109	F	F	F	F	F	F
116	F	F	F	P	F	F
128	F	F	F	F	F	F
146	F	F	F	F	F	F
147	F	F	F	F	F	F
149	P	P	P	P	P	P
150	F	F	F	F	F	F
153	F	P	P	P	F	F
156	F	F	F	F	F	F
163	F	P	F	P	P	P
167	F	F	F	F	P	F
168	F	F	F	F	F	F
170	P	P	P	P	P	P
174	F	F	F	F	F	F
200	F	F	F	F	F	F
202	F	F	F	F	F	F
204	P	P	P	P	P	P
207	P	P	P	P	P	P
211	F	F	F	F	F	F
219	F	F	F	F	F	F
228	P	P	P	P	P	P
234	F	F	F	F	F	F
239	P	P	P	P	P	F
242	F	F	F	F	F	F

Table 10 All main task outcomes, part 2/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

ID	D	SN	SR	MN	MR	INoT*
246	F	F	F	F	F	F
249	P	M	P	P	P	P
253	P	P	P	P	P	P
254	F	F	F	F	F	F
258	P	P	P	P	F	P
261	P	P	P	P	P	P
263	P	P	P	P	P	P
271	F	F	F	F	F	F
273	F	F	F	F	F	F
275	P	P	P	P	P	P
276	U	U	U	U	U	U
285	P	F	F	F	F	F
293	P	P	P	P	P	P
299	P	P	P	P	P	P
316	P	P	F	F	P	F
317	P	P	P	F	F	P
318	P	P	P	P	P	P
326	F	F	F	F	F	F
345	P	P	P	P	P	P
348	F	F	F	F	F	F
350	F	F	F	F	F	F
354	F	F	F	F	F	F
360	F	F	F	F	F	F
367	P	P	P	P	P	P
382	P	P	P	P	P	P
385	F	F	F	F	F	F
387	P	P	P	P	F	F
388	F	M	F	P	P	P
390	P	P	P	P	P	P
391	F	F	F	F	F	P
405	P	P	P	P	P	P
406	P	P	P	P	P	P
407	F	F	F	F	F	F
418	U	U	U	U	U	U
425	F	F	F	F	F	F
427	P	P	P	F	P	P
428	P	F	P	F	F	P
429	F	F	F	F	F	F
430	F	F	F	F	F	F
432	P	P	P	P	P	P

Table 11 All main task outcomes, part 3/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

ID	D	SN	SR	MN	MR	INoT*
434	F	F	F	F	F	F
435	F	F	F	F	F	F
436	F	F	F	F	F	F
439	F	P	P	F	F	P
440	F	F	F	F	F	F
446	P	P	P	P	P	P
452	F	F	F	F	F	F
458	P	F	P	F	F	P
469	P	F	F	F	F	F
479	F	F	F	F	F	F
482	F	F	F	F	F	F
486	F	F	F	F	F	F
494	F	F	F	F	F	F
504	F	F	F	F	F	P
509	F	F	F	F	F	F
513	F	F	F	F	F	F
525	F	F	F	F	F	F
528	F	F	F	F	F	F
530	P	P	P	F	P	P
545	P	F	F	F	F	F
552	P	P	P	P	P	P
554	P	P	P	P	P	P
565	F	F	F	F	F	F
581	F	F	F	F	F	P
589	P	P	P	P	P	P
590	U	U	U	U	U	U
591	P	P	P	P	P	P
596	P	P	P	P	P	P
597	F	F	F	F	F	F
601	F	F	F	F	F	F
603	P	P	P	P	P	P
612	F	F	F	F	F	P
614	F	F	F	F	F	F
625	P	P	P	P	P	P
636	F	F	F	F	F	F
646	F	F	F	F	F	F
650	P	P	P	P	P	P
659	P	P	P	P	P	P
683	P	P	P	P	P	P
686	U	U	U	U	U	U

Table 12 All main task outcomes, part 4/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

ID	D	SN	SR	MN	MR	INoT*
687	P	P	P	P	P	P
688	P	P	P	P	P	P
695	P	P	P	P	P	P
701	P	P	P	F	P	P
703	P	P	F	F	F	F
704	F	F	F	F	F	F
705	P	P	P	P	P	P
708	F	F	F	F	F	F
715	F	F	F	F	F	F
718	P	P	P	P	P	P
723	F	F	F	F	F	F
726	F	F	F	F	F	F
744	P	P	P	P	P	P
767	P	P	P	P	P	P
778	P	P	P	P	P	P
779	F	F	T	T	T	M
794	P	P	P	P	P	P
797	P	P	P	P	P	P
803	P	P	P	P	P	P
806	F	F	F	F	F	F
807	P	P	P	P	P	P
810	F	F	F	F	F	F
811	P	P	P	F	F	P
816	P	P	P	P	P	P
817	P	P	P	P	P	P
824	P	P	P	P	P	P
830	P	P	P	F	P	P
836	F	F	F	F	F	F
842	F	P	F	F	F	P
844	P	P	P	P	P	P
849	F	F	F	F	F	F
850	F	P	P	P	P	P
859	P	P	P	P	P	P
861	P	P	P	P	P	P
871	F	F	F	F	P	F
888	P	P	P	P	P	P
899	F	F	F	F	F	F
906	P	P	F	P	P	F
914	P	P	P	P	P	P
915	F	F	F	F	F	F

Table 13 All main task outcomes, part 5/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

ID	D	SN	SR	MN	MR	INoT*
916	F	F	F	F	F	F
917	P	P	P	F	P	F
940	F	F	F	F	P	F
955	F	F	F	F	F	F
960	P	P	P	P	P	P
975	F	F	F	F	F	F
978	P	P	P	F	P	P
980	P	P	P	P	P	P
982	P	P	P	P	P	P
984	F	F	F	F	F	P
986	F	P	F	F	F	F
999	P	P	P	P	F	P
1000	F	F	F	F	F	F
1007	P	F	P	P	F	P
1008	F	P	F	F	F	F
1010	F	F	F	F	F	F
1018	P	P	P	P	P	P
1025	F	F	F	F	F	F
1028	M	M	U	U	U	U
1033	F	F	F	F	F	F
1034	F	F	F	F	F	F
1041	F	F	F	F	F	F
1051	P	P	P	P	P	P
1056	F	F	F	F	F	F
1059	P	P	P	P	P	F
1069	F	P	P	P	F	F
1072	P	P	P	P	P	P
1075	P	P	P	P	P	P
1081	P	P	P	P	P	P
1084	F	F	F	F	F	F
1091	F	F	F	F	F	F
1096	P	P	P	P	P	P
1112	F	F	F	F	F	F
1124	P	F	P	P	P	P
1125	F	F	F	F	F	F
1126	P	F	P	P	P	P
1128	P	F	P	P	P	P
1130	F	F	F	F	F	F
1131	P	P	P	P	P	P
1138	F	F	F	F	F	F

C Complete development task outcomes

Table 14 Every development task and both repeats. Each pair of letters gives repeats 2 and 3 in order: P = pass, F = fail, U = unavailable. These are original test statuses after the reference eligibility gate; disputed tests have not been deleted.

ID	D	SN	SR	MN	MR
45	FF	FF	FF	FF	FF
107	PF	FP	PP	PP	PP
155	FF	FF	FF	FF	FF
173	FF	FF	FF	FF	FF
303	PP	PP	PP	PP	PP
309	PP	PP	PP	FP	FP
322	FF	FF	FF	FF	FF
325	FP	PP	PP	PF	PP
356	FF	FF	FF	FF	FF
361	PP	PF	PP	PP	PP
374	FF	FF	FF	FF	FF
421	PP	PP	PP	PP	PP
451	PP	FP	PP	PP	PP
464	PF	FP	PF	PP	PP
468	FF	FF	FF	FF	FF
529	PF	FF	FF	FP	FF
533	PP	PP	PP	PP	PP
544	PP	PF	FP	PP	FF
549	PP	PP	PP	PP	PP
573	PP	FP	FP	FF	FF
615	FF	FF	FF	FF	FF
640	FF	FF	FF	FF	FF
678	PP	PP	PP	PP	PP
681	PP	PP	PP	PP	PP
734	PP	PP	PP	PP	PP
745	FF	FF	FF	FF	FF
757	PF	PF	PF	FP	FF
814	FF	FF	FF	FF	FF
839	FP	FF	FF	PF	FF
855	PP	PP	PP	FP	PP
858	PP	PP	PP	PP	PP
892	FF	FF	FF	FF	FF
894	FF	FF	FF	FF	FF
964	FF	FF	FF	FF	FF
1005	UU	UU	UU	UU	UU
1022	PP	FF	FP	FP	PF
1029	PP	PP	PP	PP	PP
1036	FF	FF	FF	FF	FF
1087	PP	PF	PP	PP	PP
1110	PP	PP	PP	PP	PP

D Historical evidence and calibration

The earlier 240 Flash-Lite records [8] cover 20 tasks, four target lengths, three architectures and one seed, not 240 independent tasks. Saved quality labels are all one and full solutions are missing: arithmetic is checkable, correctness cannot be replayed. B3 has an extra rerun in 48/80 records; its final labels are not single-attempt pass@1. Its comparison with B2 mixes roles, calls, reruns and context selection. Relative to the simple B0 solver it costs 10.3–85.0% more at the three shorter contexts; the longest-context saving coincides with input selection. These records motivate the replacement but do not enter its estimates.

The preceding eight-task pilot generated 40 candidates in 72 turns at USD 0.7417401 API-equivalent valuation. On seven evaluable tasks, direct solving passed 4/7, single-call roles 3/7 and three-call roles 2/7. The extension uses fresh responses. An earlier CLI 0.144.1 attempt was rejected after an unexpected planning tool appeared; all 21 submitted turns and known USD 0.2493285 valuation remain archived. Rejected responses are never spliced into the valid matrix.

References

- [1] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. <https://arxiv.org/abs/2507.08664v1>.
- [2] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. <https://arxiv.org/abs/2604.02460v2>.
- [3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. <https://arxiv.org/abs/2407.01489>.
- [4] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. <https://arxiv.org/abs/2405.15793>.
- [5] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. <https://arxiv.org/abs/2406.15877>.
- [6] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. <https://arxiv.org/abs/2310.06770>.
- [7] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. <https://www.swebench.com/SWE-bench/guides/evaluation/>.
- [8] D. Privezentsev. INoT_Research. Historical implementation, commit 010211ee5775185e8330ab6cde06e912c2adcb9e. https://github.com/daniil-novel/INoT_Research.

- [9] D. Privezentsev. Article-INoT: manuscript, historical artifacts and compression-free revision protocol. 2026. <https://github.com/daniil-novel/article-INoT>.
- [10] OpenAI. GPT-5.4 mini model documentation. Accessed 8 September 2026. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>.
- [11] OpenAI. API pricing, standard text-token rates. Accessed 8 September 2026. <https://developers.openai.com/api/docs/pricing>.
- [12] OpenAI. Codex non-interactive mode and JSONL event stream. Accessed 8 September 2026. <https://developers.openai.com/codex/noninteractive>.
- [13] M. Zheng, J. Pei, L. Logeswaran, M. Lee, and D. Jurgens. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. Findings of EMNLP, pp. 15126–15154, 2024. <https://aclanthology.org/2024.findings-emnlp.888/>.
- [14] P. H. Luz de Araujo, P. Röttger, D. Hovy, and B. Roth. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. EMNLP, pp. 26857–26886, 2025. <https://aclanthology.org/2025.emnlp-main.1364/>.
- [15] H. K. Choi, X. Zhu, and S. Li. Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? NeurIPS, 2025; arXiv:2508.17536v2. <https://arxiv.org/abs/2508.17536v2>.
- [16] F. V. Wunderlich, L. B. Kaesberg, J. P. Wahle, T. Ruas, and B. Gipp. Multi-Agent Reasoning Improves Compute Efficiency: Pareto-Optimal Test-Time Scaling. ACL Student Research Workshop, pp. 1–14, 2026. <https://aclanthology.org/2026.acl-srw.1/>.